

scrape_total

June 23, 2026

1 1. Scraping del dataset europeo

Nowtricity non ha API pubbliche e carica i dati in modo asincrono. Fare scraping classico con `requests` è inutile perché restituisce solo una pagina vuota. Invece di pilotare il browser per cliccare mese per mese (che ci metterebbe una vita ed è pronò a crash), questo script fa tre cose: 1. Carica la pagina principale della singola nazione per recuperare gli ID delle query nascoste. 2. Interroga direttamente gli endpoint AJAX per scaricare i JSON grezzi. 3. Parsa l'HTML al volo, tiene solo i valori numerici e salva un CSV per ogni nazione.

```
[ ]: import os
import glob
import time
import json
import random
import pandas as pd
from bs4 import BeautifulSoup
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Dizionario dei codici ISO e relativi slug delle nazioni
Countries = {
    "AT": "austria", "BE": "belgium", "BG": "bulgaria", "HR": "croatia",
    "CZ": "czech-republic", "DK": "denmark", "EE": "estonia", "FI": "finland",
    "FR": "france", "DE": "germany", "GR": "greece", "HU": "hungary",
    "IT": "italy", "LV": "latvia", "LT": "lithuania", "NL": "netherlands",
    "NO": "norway", "PL": "poland", "PT": "portugal", "RO": "romania",
    "RS": "serbia", "SK": "slovakia", "SI": "slovenia", "ES": "spain",
    "SE": "sweden", "GB": "united-kingdom",
}

# Creazione delle directory di output e archiviazione dati
os.makedirs("../data", exist_ok=True)
os.makedirs("../outputs", exist_ok=True)
```

```

[ ]: def scrape_country(driver, country_slug, country_code):

    print(f"\n[{country_code}] Avvio estrazione per {country_slug.upper()}")

    base_url = f"https://nowtricity.com/country/{country_slug}/"
    try:
        driver.get(base_url)
        # Recupero i blocchi annuali per capire quali mesi scaricare
        years_elements = WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CLASS_NAME,
↪ "js-load-year-data"))
        )
    except Exception as e:
        print(f"[{country_code}] Fallimento nel caricamento della pagina: {e}")
        return False

    all_queries = []
    for year_element in years_elements:
        year_str = year_element.get_attribute("data-year")
        if not year_str or len(year_str) != 4:
            continue
        for month in range(1, 13):
            all_queries.append(f"{year_str}{month:02d}")

    print(f"[{country_code}] Rilevate {len(all_queries)} query mensili. Inizio
↪ download...")

    raw_data_list = []
    # Scarico i payload JSON interrogando direttamente l'API del sito
    for query_id in all_queries:
        ajax_url = f"https://www.nowtricity.com/ajax.php?
↪ c=Pais&m=svg&country={country_slug}&type=month&query={query_id}"

        try:
            driver.get(ajax_url)
            json_text = driver.find_element(By.TAG_NAME, "body").text
            payload = json.loads(json_text)
            raw_data_list.append({
                "mese": query_id,
                "json_payload": payload
            })
        except Exception as e:
            print(f"[{country_code}] Errore nell'estrazione di {query_id}.
↪ Salto il record.")

            time.sleep(random.uniform(3.5, 6.5)) # Pausa Randomica per eludere
↪ l'anti-bot

```

```

# Salvataggio dei dati grezzi
json_path = f"../data/nowtricity_raw_{country_code.lower()}.json"
with open(json_path, "w", encoding="utf-8") as f:
    json.dump(raw_data_list, f, ensure_ascii=False, indent=4)

clean_records = []
# Pulisco l'HTML integrato nei JSON e isolo le metriche
for element in raw_data_list:
    month_id = element["mese"]
    payload = element["json_payload"]
    html_content = payload.get("html", "") if isinstance(payload, dict)
    else ""

    soup = BeautifulSoup(html_content, "html.parser")
    try:
        co2_raw = soup.find("text", class_="emissions").text.strip()
        renewable_raw = soup.find("div",
class_="emissions-graph-renewable").text.strip()
        total_raw = soup.find("div", class_="emissions-graph-total").text.
strip()

        record = {
            "country_code": country_code,
            "country_name": country_slug,
            "month_id": month_id,
            "co2_g_kwh": float(co2_raw.replace(" g", "").strip()) if
co2_raw.replace(" g", "").replace('.', '', 1).strip().isdigit() else None,
            "renewable_perc": float(renewable_raw.replace("% renewable",
"").strip()) if renewable_raw.replace("% renewable", "").replace('.', '', 1).
strip().isdigit() else None,
            "total_twh": float(total_raw.replace(" TWh total", "").strip())
if total_raw.replace(" TWh total", "").replace('.', '', 1).strip().isdigit()
else None
        }

        energy_voices = soup.find_all("div", class_="entry")
        for voice in energy_voices:
            text = voice.text.strip()
            if "(" in text and "%" in text:
                last_open = text.rfind("(")
                source_name = text[:last_open].strip().lower().replace(" ",
"").replace("(", "").replace(")", "")
                perc_val = text[last_open+1:].replace("%", "").
replace("%", "").strip()

```

```

        record[f"source_{source_name}_perc"] = float(perc_val) if
↳perc_val.replace('.', '', 1).isdigit() else 0.0

        clean_records.append(record)
    except AttributeError:
        continue

    # Esportazione dei record puliti in formato CSV dedicato
    if clean_records:
        df = pd.DataFrame(clean_records)
        df.fillna(0, inplace=True)
        csv_path = f"../outputs/nowtricity_{country_code.lower()}.csv"
        df.to_csv(csv_path, index=False, encoding="utf-8")
        print(f"[{country_code}] Salvati con successo {len(df)} record in
↳{csv_path}")
        return True

    return False

```

```

[ ]: def is_already_processed(country_code):
    """ Controlla se abbiamo già il CSV, utile per riprendere il ciclo in caso
↳di crash """
    csv_path = f"../outputs/nowtricity_{country_code.lower()}.csv"
    return os.path.exists(csv_path)

```

1.0.1 1.0.1 Gestione anti-ban e unione dei dati

Sparare richieste a raffica su un server esterno fa scattare subito il ban dell'IP. Per evitarlo, il loop principale ha un paio di accortezze: - **Skip dei file esistenti**: Se la connessione cade a metà, lo script legge i CSV già scaricati e riparte da dove si era fermato. - **Pausa Random**: Aspetta dai 15 ai 25 secondi tra una nazione e l'altra per simulare tempi umani.

Alla fine, lo script prende tutti i CSV nazionali e li unisce in un unico master dataset.

```

[ ]: # Inizializzazione e configurazione del WebDriver Selenium
chrome_options = Options()
user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
↳(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
chrome_options.add_argument(f"user-agent={user_agent}")

driver = webdriver.Chrome(options=chrome_options)

try:
    for iso_code, slug in Countries.items():
        if is_already_processed(iso_code):
            print(f"Salto {iso_code} ({slug}) - Il file CSV esiste già.")
            continue

```

```

        success = scrape_country(driver, slug, iso_code)

        if success:
            # Pausa estesa tra le sessioni per prevenire il blocco dell'IP
            ↪(Rate Limiting)
            cooldown = random.uniform(15, 25)
            print(f"[{iso_code}] Attesa di {cooldown:.1f} secondi per prevenire
            ↪il ban temporaneo...")
            time.sleep(cooldown)
        else:
            print(f"[{iso_code}] Estrazione fallita. Passaggio al target
            ↪successivo.")

    finally:
        driver.quit()
        print("\nESECUZIONE DEL CICLO PRINCIPALE TERMINATA.")

```

```

[ ]: print("Avvio del processo di aggregazione dei dataset...")

# Identificazione di tutti i file CSV parziali
all_csv_files = glob.glob("../outputs/nowtricity_*.csv")

if not all_csv_files:
    print("Nessun file CSV rilevato per l'unione.")
else:
    df_list = []

    for file in all_csv_files:
        # Salto il master se lancio la cella due volte
        if "master_dataset.csv" in file:
            continue
        try:
            temp_df = pd.read_csv(file)
            df_list.append(temp_df)
        except Exception as e:
            print(f"Fallimento nella lettura del file {file}: {e}")

    if df_list:
        # Concatenazione dei DataFrame in un'unica struttura dati
        master_df = pd.concat(df_list, ignore_index=True)

        # Normalizzazione dei valori mancanti (NaN)
        master_df.fillna(0, inplace=True)

        master_df.sort_values(by=["country_code", "month_id"], inplace=True)

        # Salvataggio del dataset definitivo

```

```
final_path = "../outputs/europe_nowtricity_master_dataset.csv"
master_df.to_csv(final_path, index=False, encoding="utf-8")

print(f" Aggregazione completata. Master dataset salvato in:␣
↪{final_path}")
print(f" Dimensioni totali: {master_df.shape[0]} righe, {master_df.
↪shape[1]} colonne.")
```